ToolGuard: Measuring the Effect of Validation Gates and Bounded Retries in a Synthetic Tool-Use Workflow

Article status: research draft; update results from a verified run before publication. **Release:** 0.1.0 (21 October 2026)

Abstract

Tool-using AI agents interact with software through actions that can fail transiently, contain invalid arguments, or violate workflow prerequisites. This reproducibility project provides a lightweight synthetic benchmark for separating two execution-control mechanisms: pre-execution validation/prerequisite gating and bounded retries. It compares four scripted policies across controlled tool-failure rates using generated ticket-triage tasks. The package reports task success, invalid-action rate, tool-call overhead, gate interventions, repairs, premature closure attempts, and budget exhaustion. An optional adapter can run a local Ollama model against the same workflow. The benchmark is intended as a transparent teaching and research baseline, not as evidence of production reliability or general LLM reasoning ability.

Motivation and research question

Current UK institutional research pages explicitly describe agentic systems that must act safely and explainably in dynamic environments. Ulster University's Computing research theme includes trust-aware agents, adaptive decision-making, and agentic process optimisation. The University of Glasgow's Computing Science research environment spans machine learning, autonomous systems, HCI, and research degrees; its application guidance asks research proposals to demonstrate originality and methodological rigour. UCL's foundational-AI studentship identifies sequential decision-making and computational experiments as relevant directions. These pages motivate studying the execution layer of tool-using agents, but do not establish that this particular question is novel.

Research question: Under a fixed synthetic workflow, how do validation gates, prerequisite checks, and bounded retries affect task completion, invalid tool actions, and tool-call overhead as transient tool failures increase?

Hypotheses

- **H1:** Validation and prerequisite gating will reduce invalid actions and premature closure attempts.
- **H2:** Bounded retries will improve completion when tools fail transiently, at the cost of additional tool calls.
- **H3:** Combining validation with bounded retries will provide a useful reliability/overhead trade-off, although the result may depend on noise and failure rate.

These are hypotheses to test, not results assumed in advance.

Method

The benchmark generates synthetic ticket records with a category and severity. A successful workflow must inspect the ticket, set the correct priority, assign the category-matched team, add a non-empty note, and close only after prerequisites are satisfied. The benchmark introduces two controlled perturbations: argument/order noise in the proposed plan and transient tool failures. Four policies are compared: baseline, retry-only, guard-only, and guard-plus-retry. The default design uses 120 tasks, 12% plan-noise probability, tool-failure rates of 0%, 10%, and 25%, an eight-call budget, and at most two retries after an initial attempt.

All data are generated locally with a fixed seed. The default experiment uses a scripted planner to isolate the execution-control layer; it does not ask an LLM to reason. The optional Ollama experiment is a separate extension and must be reported separately. The package includes unit tests, per-episode results, aggregate metrics, run metadata, and dependency-free SVG figures.

Results from the reference run

The package was executed during preparation on 21 October 2026 using Python 3.13.5, seed 7, 120 tasks, 12% plan-noise probability, an eight-call budget, and up to two retries after the initial attempt. The tests passed (7/7). The results below are generated simulator outputs, not measurements from the user's Mac and not LLM results. Re-run the command on the target environment and retain its `run_metadata.json` before publication.

Tool-failure rate	Policy	Task success	Invalid-action rate	Mean tool calls
0%	Baseline	50.8%	8.0%	5.00
0%	Retry only	50.8%	8.0%	5.00
0%	Guard only	100.0%	0.0%	5.00
0%	Guard + retry	100.0%	0.0%	5.00
10%	Baseline	30.0%	16.3%	5.00
10%	Retry only	49.2%	7.2%	5.54
10%	Guard only	61.7%	0.0%	4.37
10%	Guard + retry	97.5%	0.0%	5.53
25%	Baseline	13.3%	20.7%	5.00
25%	Retry only	44.2%	7.0%	6.40
25%	Guard only	24.2%	0.0%	3.79
25%	Guard + retry	85.0%	0.0%	6.30

In this implementation, the guard policies removed invalid tool actions because the validator repairs the known synthetic category/severity mappings and gates premature closure. Retry-only improved completion as transient failures increased, but it did not correct argument/order noise. Guard-plus-retry had the highest completion in each tested failure condition, at the cost of more calls when tools failed. At 25% failure, 10.8% of guard-plus-retry episodes and 15.8% of retry-only episodes exhausted the call budget without success. These results are consistent with the deliberately encoded simulator mechanics; they do not establish a general claim about LLM agents or real workflows.

There is only one seed and no confidence interval in this reference run. Before a research submission, repeat across multiple seeds, report variability/uncertainty, and add additional workflows.

Recommended figures: `outputs/figures/success_rate.svg`, `tool_call_overhead.svg`, and `invalid_action_rate.svg`.

Limitations

The workflow is intentionally small, synthetic, and deterministic apart from controlled noise/failure events. The expected mapping from category/severity to team/priority is encoded by design, so the benchmark does not measure domain knowledge. The scripted planner is not an LLM and the results cannot be interpreted as a model leaderboard. Only a small number of policy variants and failure levels are included. A stronger study should run multiple seeds, report uncertainty intervals, add additional workflows, compare blocking versus repairing gates, and evaluate at least one local LLM on a larger sample.

Reproducibility

Run from the project directory:

```
python3 -m unittest discover -s tests -v
python3 run_experiment.py --tasks 120 --seed 7
python3 plot_results.py
```

The code uses the Python standard library. The run metadata records the environment and parameters. The task data are synthetic; no external dataset or real customer information is used.

Responsible reuse

This is a research/teaching benchmark, not a customer-support product. Do not connect it to live ticketing systems without a separate security, privacy, human-oversight, and safety review. Publish the simulator, generated data, and measured results together so others can reproduce the exact run. Do not claim affiliation with or endorsement by any university mentioned below.

References and current institutional context (checked 21 October 2026)

1. Ulster University. *Agentic AI for Complex Environments*. https://www.ulster.ac.uk/doctoralcollege/find-a-phd/11a-computing/1903114
2. University of Glasgow, School of Computing Science. *Studentship Information*. https://www.gla.ac.uk/schools/computing/postgraduateresearch/prospectivestudents/scholarshipsandfundingopportunities/studentshipinformation/
3. University of Glasgow, School of Computing Science. *Postgraduate research study and proposal guidance*. https://www.gla.ac.uk/schools/computing/postgraduateresearch/prospectivestudents/
4. University College London. *Fully Funded PhD Studentship in Foundational AI*. https://www.ucl.ac.uk/engineering/fully-funded-phd-studentship-foundational-ai

These institutional pages support the choice of a research direction; they are not peer-reviewed literature and do not replace a formal literature review. Add relevant peer-reviewed work on LLM agents, tool-use evaluation, software verification, and retry/validation strategies before submitting the work as a research paper.